1.5em 0pt

Scuola universitaria professionale
della Svizzera italiana

SUPSI

Masters Degree in
Computer Science

MSE Project Report

Variable Partitioned Bayesian Optimization for Feature Selection in Sustainability Assessment in GRETA

Supervisor

Prof. Giuseppe Landolfi

Student

Saad Raza Hussain Shafi

Academic year 2023/2025

Contents

Abstract

This project builds an AI workflow that identifies which GRETA customisation inputs most affect sustainability outcomes and cost. GRETA estimates impacts such as climate change, ecosystem effects, resource use, and cost for manufacturing processes. GRETA was used to estimate climate change, ecosystem quality, resource use, and cost for manufacturing processes and products. Seven GRETA datasets were cleaned and merged into a single table with about 164 thousand records and 70 variables. Sixteen outputs were grouped by correlation into four composite targets, namely Human health total, Ecosystem quality total, Resources total, and Cost. Predictors were screened with distance correlation and Pearson correlation. A feedforward neural network was trained, and the feature selection threshold together with network capacity was tuned by variable partitioned Bayesian optimization using a multi target objective with worst case scalarization. On the GRETA data, forty predictive inputs were retained. Median relative error of 0.1475 for Human health total, 0.0624 for Ecosystem quality total, 0.2347 for Resources total, and 0.0028 for Cost was achieved, with convergence in about ten trials. GridSearchCV was observed to reach slightly lower error at the expense of roughly four times more evaluations. The workflow was designed to be reproducible and to be integrated into GRETA to provide actionable sensitivity and feature importance for design and process decisions.

1 Introduction

Manufacturing must reduce environmental impacts while controlling costs. Decisions on design, sourcing, and process settings depend on understanding which inputs most change outcomes. Life cycle assessment tools support this need, yet the number of controllable parameters is large, interactions are complex, and manual sensitivity studies are slow.

GRETA estimates sustainability impacts and cost for products and processes, given inputs such as production location, energy mixes, materials, transportation, and process parameters. The central objective in this work is the identification of the GRETA customisation inputs that most influence impacts and cost, with accuracy that is robust and with computation that remains tractable. The outputs are rankings and sensitivity information that can be integrated into GRETA to guide users toward effective changes.

The study is framed by four research questions. First, which input parameters most influence each impact category and cost across scenarios. Second, how many features are required to achieve stable accuracy with limited compute. Third, how the proposed method compares with a standard search baseline in accuracy. Fourth, how the methods can be parallelized to handle big data problems.

A data pipeline is constructed to consolidate GRETA datasets, align formats and units, and prepare targets and predictors. Impact outputs are grouped by correlation into four composite targets that reflect GRETA reporting, namely Human health total, Ecosystem quality total, Resources total, and Cost. Predictors are screened using distance correlation and Pearson correlation to remove weak and duplicated signals. A feedforward neural network is trained, and its capacity together with the feature selection threshold is tuned using variable partitioned Bayesian optimization with a multi target objective that minimizes median relative error under a worst-case scalarization.

Validation covers grouped cross validation, a comparison against GridSearchCV, sensitivity to the selection threshold, and stability across random seeds. Assumptions and limits are stated. GRETA impact calculations are treated as ground truth for modeling purposes, causal claims are not made, and results depend on the chosen model class and objective.

The contributions are the following. A reproducible data pipeline for GRETA. A compact correlation based filter that reduces the predictor set while preserving accuracy. A tuning strategy that delivers near optimal performance with few evaluations. A validation protocol that quantifies accuracy, efficiency, and stability. An integration plan that surfaces ranked drivers, enables scenario exploration, and supports design and process decisions.

The remainder of the document presents background and related work, the dataset and preprocessing, the modeling and optimization approach, the validation and results, and the conclusion.

2 Literature Review

2.1 AI in Sustainability and Life Cycle Assessment (LCA)

Recent studies have explored how machine learning can complement Life Cycle Assessment (LCA) by identifying key parameters driving environmental impacts. For example, Muller et al. (2021) demonstrated the use of machine learning for predicting life cycle Carbon Dioxide emissions in the automotive sector, showing that input material composition was the dominant driver [12]. Cucurachi et al. (2019) reviewed the integration of AI into LCA and highlighted its potential for handling high-dimensional input data and uncertainty[6]. Bello et al. (2020) applied random forests to construction LCA, identifying energy mix and transportation distances as key sustainability hotspots[1]. Explainable AI (XAI) methods such as SHAP and LIME have been increasingly applied to sustainability analytics, enabling stakeholders to interpret which parameters contribute most to environmental outcomes [17].

These works reinforce the relevance of applying feature selection and optimization methods, such as those proposed in this project, to GRETA's sustainability datasets.

2.2 Feature Selection

For any machine learning model, the selection of features is based on their importance to the prediction of the required target variable [10]. This importance can be measured in a number of ways. Based on this, feature selection can be divided into three categories:

Filter methods where variables are measured using some ranking criterion independent of the learning algorithm [9]. This criterion is based on the correlation among the variables and responses. One of the most popular methods is Pearson's correlation. It is defined as the covariance of the variables that are to be compared, divided by their standard deviations [2]. Pearson's correlation does not imply a causal relationship between the two variables, and there may be a hidden variable that serves as the underlying cause of the change. The significance of a correlation is measured using a p-value in Pearson correlation. the p-value cutoff can be set at 0.01 or 0.05, where a calculated p-value lower than the cutoff implies statistical significance of the relationship. The Scipy function in Python offers a Pearson function that computes the Pearson correlation and produces a two-tailed P value. Pearson's value is good for bivariate normally distributed datasets. The values range from -1 to 1, where 1 indicates a perfect match, while -1 indicates a perfect negative correlation. This way, it can suggest direction of the relationship. The value 0 indicates no linear relationship. However, there are weaknesses of Pearson's correlation, like its inability to detect or explain correlations for non-linear relationships, ordinal or non-normal datasets.

Distance correlation is another method for measuring the nonlinear relationships among the variables. Proposed by Szekely and Rizzo, the distance correlation (R (X, Y)) measures how independent the two variables are from one another using a distance formula [16].

This correlation measure is zero if the two variables are independent. The closer to zero, the less they depend on each other. The threshold for $R(X,Y) \leq 0.1$ is found to be a strong indication of two variables being independent. The method has also been found robust to noise as compared to other methods. The advantages of filter methods are that they are computationally light and do not depend on the learning

$$\mathcal{R}_n^2(X,Y) = \frac{\nu_n^2(X,Y)}{\sqrt{\nu_n^2(X)\nu_n^2(X)}}$$

Figure 2.1: dcor formula

algorithm. Disadvantages of filter methods are their inability to reach an optimal feature set due to a concrete threshold between correlated and uncorrelated variables. Moreover, the correlation is influenced by functional dependence and level of noise, and sample size.

The advantages of filter methods are that they are computationally light and do not depend on the learning algorithm. Disadvantages of filter methods are their inability to reach an optimal feature set due to a concrete threshold between correlated and uncorrelated variables. Moreover, the correlation is influenced by functional dependence and level of noise, and sample size.

Wrapper Methods: Wrapper methods evaluate subsets of variables by training predictive models, making them more accurate but computationally expensive. As the feature subset search problem is NP-hard, heuristics such as greedy search are often applied. While effective for small to medium problems, these approaches are impractical for large-scale feature spaces.

Embedded Methods: Embedded methods represent a compromise between filter and wrapper methods. They incorporate feature selection during the model training process, for example, through regularization techniques such as LASSO, thereby combining efficiency with improved accuracy.

2.3 Hyperparameter Optimization

Hyperparameter selection is also another important factor affecting the performance of the machine learning models [5]. These hyperparameters can range from learning rate, regularization constants, and non-linearity type. The selection of hyperparameters using traditional methods such as grid search and random search is very time-consuming and difficult, requiring the complete coverage of the search space.

One technique to reduce time and resource consumption is to use a surrogate model that mimics the behavior of the real complex model. The optimization algorithm using a surrogate model maps the error dependence of hyperparameters iteratively on the dataset. This iterative process estimates the error curve. A Multilayer Perceptron (MLP) is a common surrogate model used for feature and hyperparameter selection.

2.4 Bayesian Optimization

Bayesian Optimization (BO) is a powerful framework for optimizing expensive black-box functions [3]. BO typically employs Gaussian Processes (GPs) as surrogate models to characterize the distribution of possible functions, defined by a mean and variance over the search domain [7].

Algorithm 1 Standard Bayesian Optimization (S-BO)

Input: Acquisition function parameter κ, number of iterations L, initial dataset $\mathcal{D}_0$
Output: Optimized dataset $\mathcal{D}_L$ and surrogate model $\hat{f}_L$
begin
  Train the GP surrogate model $\hat{f}_0$ using the initial dataset $\mathcal{D}_0$ Compute the acquisition function AF_0^f
  for $\ell = 1$ **to** L **do**
    Compute next experiment point: $x_{\ell+1} \leftarrow \arg\min_{x \in \mathcal{X}} AF_\ell^f(x;\kappa)$
    Evaluate the black-box function at $x_{\ell+1}$ to obtain $f_{\ell+1}$
    Update dataset: $\mathcal{D}_{\ell+1} \leftarrow \mathcal{D}_\ell \cup \{(x_{\ell+1}, f_{\ell+1})\}$
    Retrain the GP surrogate model $\hat{f}_{\ell+1}$ using $\mathcal{D}_{\ell+1}$ Update the acquisition function $AF_{\ell+1}^f$

An **acquisition function** then determines the next evaluation point, balancing exploration by sampling from high uncertainty and exploitation by sampling from regions of high performance [11].

Common acquisition strategies include Expected Improvement (EI) and Probability of Improvement (PI) [13]. For minimization, let the surrogate posterior be $Y(x) \sim \mathcal{N}(\mu(x), \sigma^2(x))$ with the current best $f^\star$. EI balances exploitation and exploration via

$$\mathrm{EI}(x) = \big(f^\star - \mu(x)\big)\,\Phi(z) + \sigma(x)\,\phi(z), \quad z = \frac{f^\star - \mu(x)}{\sigma(x)}.$$

PI selects points maximizing the probability of improvement,

$$\mathrm{PI}(x) = \Phi\left(\frac{f^\star - \xi - \mu(x)}{\sigma(x)}\right),$$

where the jitter $\xi > 0$ tempers greediness and promotes exploration. In practice, EI is often preferred for its sensitivity to improvement magnitude, while PI can over-exploit unless carefully tuned.

BO is particularly well-suited for ML applications as it accounts for stochastic variability in training and testing errors. In this study, BO is employed both for hyperparameter optimization (e.g., number of neurons) and for selecting thresholds in distance correlation-based feature filtering.

2.5 Parallel Experimentation and Bayesian Optimization

Parallel experimentation accelerates the search process by evaluating multiple candidate configurations simultaneously. While this reduces runtime, large design spaces can diminish efficiency due to redundant or clustered sampling. A basic approach to DoE is screening, in which experiments are performed at different points in a discretized grid [14]. However, this approach does not scale at large and causes the waste of resources.

Advanced DoE techniques maximize the value by reducing the number of experiments and resources. The use of Bayesian experimental design offers another great method to improve the value of experimentation. The BO techniques are scalable and sample efficient. Also, BO can offer to tune hyperparameters and reinforcement learning, and experimental design [15]. BO can also accommodate discrete and continuous design variables and constraints for additional domain knowledge and restricting the design space [4].

The parallelization of BO is done using methods like hyperspace partitioning, batch Bayesian optimization, and NxMCMC and AF optimization over a set of exploratory designs. These approaches work better than sequential BO but are limited in parallelization, leading to wasting resources by redundant experimentations and getting trapped in local solutions.

Hyperspace Partitioning: This approach divides the search space into multiple subspaces, each modeled by an independent Gaussian Process (GP) [8]. A parameter $\phi \in [0, 1]$ controls the overlap between partitions: higher values of ϕ enable greater communication across subspaces, resulting in smoother convergence, whereas lower values improve scalability at the cost of potential discontinuities between partitions.

Advantages of hyperspace partitioning include scalability to high-dimensional spaces, easy implementation, and enabling different GPs for different spaces, allowing for to capture of local trends. Moreover, the algorithms are more extensive in exploration, and hence, a global solution is guaranteed. However, the disadvantages of this technique involve the equal size of the partitions, which might render it unable to capture complex functions. Also, the tuning of k is difficult. Automatic partitioning and tuning of hyperparameters can decrease the parallelization.

2.6 Informed Partitioning and Multi-Fidelity BO

Recent advances suggest that incorporating system structure into BO can enhance parallel optimization. González and Avola [8] proposed two methods:

1. **Level-Set Partitioning:** The design space is divided using reference functions (e.g., low-fidelity models) that approximate the performance landscape. **Multi-Fidelity BO (MFBO)** leverages these approximations to prioritize high-value regions, reducing the need for exhaustive sampling.

2. **Variable Partitioning:** The design space is divided into subunits based on prior knowledge of the system structure. This enables finer control over sampling and enhances exploration of complex, high-dimensional domains. Extensions such as **Ref-BO**, which replace surrogate low-fidelity models with empirically derived physical or analytical reference functions, further improve convergence and robustness.

3 Methodology

For the project, firstly, the data has been cleaned and processed into a suitable format. Then, the target and feature columns are identified using dcor and Pearson's correlation scores. Selected features are then forwarded to the Bayesian variable partitioning loop. The objective of our optimization procedure is to identify hyperparameter configurations that yield robust predictive performance in a **multi-target regression problem**. Specifically, we consider a neural network regressor trained on four correlated output variables. Each candidate configuration is evaluated by computing the prediction error for each output separately, resulting in a vector-valued objective function. To enable Bayesian optimization, these multi-output errors are scalarized by taking the minimum performance across all targets, thereby ensuring balanced accuracy and preventing the optimization from favoring only the easiest-to-predict outputs. The final objective function can thus be interpreted as a **partitioned, worst-case regression error metric**, which guides the optimizer toward hyperparameter settings that improve generalization across all targets simultaneously.

3.0.1 Objective Function

The objective function in our Variable Partitioned Bayesian Optimization (VPBO) framework is defined as a **multi-target regression performance measure** that evaluates the predictive accuracy of a neural network model under varying hyperparameter configurations. Let $\mathbf{X} \in \mathbb{R}^{n \times d}$ and $\mathbf{Y} \in \mathbb{R}^{n \times m}$ denote the input and output datasets, where n is the number of samples, d is the number of input features, and m is the number of output variables (targets). In our setting $m = 4$.

The model under optimization is a feedforward multi-layer perceptron (MLP) parameterized by a vector of hyperparameters $\boldsymbol{\theta} \in \mathbb{R}^p$, with $p = 2$ (corresponding to the threshold parameter and the hidden layer size).

Partitioned Objective Definition

To account for the multi-output nature of the task, we define **split-specific sub-objectives** corresponding to each target variable y_j, where $j \in \{1, \ldots, m\}$. For a given candidate hyperparameter vector $\boldsymbol{\theta}$, the sub-objective is defined as the median relative error (MSE) between the true and predicted values of the target j:

$$f_j(\boldsymbol{\theta}) = \text{MRE} = \text{median}_i \left(\frac{|y_i - \hat{y}_i|}{|y_i|} \right),$$

where $\hat{y}_{ij}(\mathbf{x}_i; \boldsymbol{\theta})$ denotes the model prediction for the j-th target of the sample $\mathbf{x}_i$.

Thus, the complete objective function is vector-valued:

$$\mathbf{f}(\boldsymbol{\theta}) = \big(f_1(\boldsymbol{\theta}), f_2(\boldsymbol{\theta}), \ldots, f_m(\boldsymbol{\theta})\big).$$

The surrogate model in this case is a feedforward multi-layer perceptron (MLP) neural network, trained using the Levenberg-Marquardt algorithm. The idea behind feedforward feature engineering is that whenever any variable exceeds a specified threshold of a correlation metric, it is included in the training process. Each variable is tested individually against the threshold, which reduces the number of hyperparameters (such as kernel type or regularization) compared to using SVM or GP-based models.

The algorithm proceeds as follows:

1. Train iteratively to cover the hyperparameter and feature selection space uniformly.
2. Use the optimizer to evaluate the error space and determine new hyperparameters for the next training iteration.
3. Repeat until the results converge.

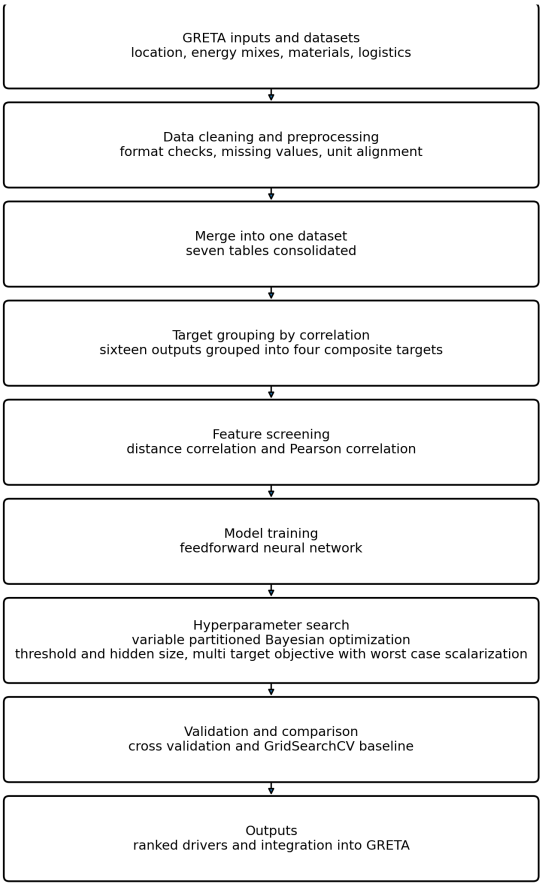


Figure 3.1: Process Flow

4 Data Analysis

4.1 Dataset Description

The dataset contains data obtained from the GRETA lifecycle assessment tool. The data set used in this study consisted of seven distinct data files, each representing a portion of the overall data collection process. These files were provided in pickle format and imported as individual data frames (df0 through df6).

4.2 Preprocessing and Cleaning

Initial exploration revealed inconsistencies in some files, including column misalignments and shifted entries. These were addressed through:

- Truncating affected rows to preserve data integrity
- Standardizing column names across all dataframes based on the reference (df0)
- Ensuring consistent data types: all non-numeric columns were converted to strings prior to encoding. Label Encoding has been used for transforming these non-numerical columns, and then to floats for modeling purposes
- Removing anomalous entries and outliers identified during exploratory analysis

After thorough inspection and preprocessing, like removing misaligned rows, the data frames were concatenated to form a single unified dataset for subsequent analysis. The final combined dataset contains over 164,148 rows and 70 columns, providing a clean and comprehensive dataset suitable for advanced statistical and machine learning analysis. The dataset was saved in CSV format to ensure reproducibility and ease of access.

4.3 Exploratory Data Analysis (EDA)

4.4 Identification of Features and Target Variables

After importing the encoded dataset, we have explored the dataset to divide it into features and target columns. Each record in the dataset represents a unique instance of a manufacturing process, capturing both input features and output targets relevant to cost estimation and environmental impact assessment. The columns are categorized as follows:

4.4.1 Features:

These include both categorical and continuous variables describing process parameters, material properties, machine specifications, transportation details, and other operational characteristics. Examples include: surfaceProcessingLocationCavity, hotRunner, manufacturingCost, designTime, weightMould, transportCost, injectedMaterial_product, machineName, among others.

4.4.2 Targets:

The dataset contains 16 target columns focused on cost and environmental impact metrics. These include: Cost, photochemical oxidation, ionizing radiation, respiratory effects, human toxicity, ozone layer depletion, terrestrial ecotoxicity, aquatic ecotoxicity, land occupation, acidification & and nutrification, mineral extraction, non-renewable energy use, total resources, climate change, total climate change impact

4.4.3 Correlation Analysis among Variables and Grouping Target Variables

The correlation analysis is performed to analyze the relationship among the variables. We have used distance correlation for this purpose. The correlation analysis reveals that some of the variables are highly

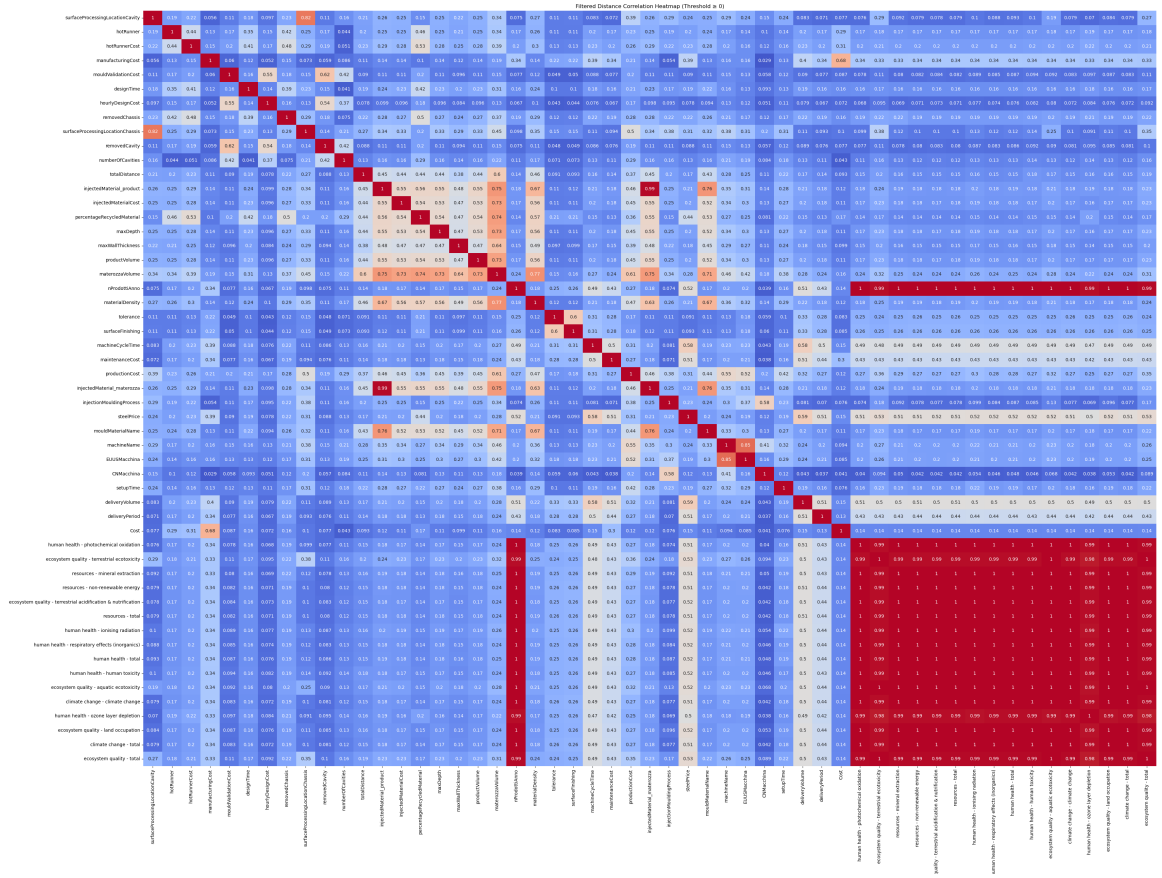


Figure 4.1: Heatmap using dCor Correlation where 0 indicates no relationship and 1 indicates duplicate/highly correlated

correlated. These include target variables: photochemical oxidation, ionizing radiation, respiratory effects, human toxicity, and ozone layer depletion. Hence, it has been decided to keep one representative target variable from the 16 initial.

The final target columns are:

- **Human health - total:** representing photochemical oxidation, ionising radiation, respiratory effects, human toxicity, and ozone layer depletion.
- **Ecosystem quality - total:** representing terrestrial ecotoxicity, aquatic ecotoxicity, land occupation, acidification & , nutrification, and total climate change impact.
- **Resources - total:** representing mineral extraction, non-renewable energy use, and total resources.
- **Cost**

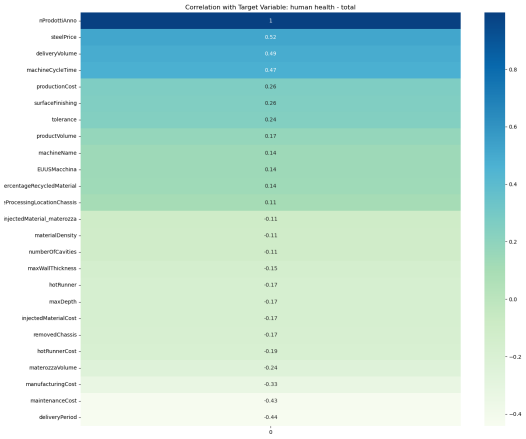
4.4.4 Correlation Analysis among Features and Target Variables

The selected target variables were then correlated individually with the features. In most literature, a common threshold for selecting relevant variables is between 0.5 and 0.7; however, applying this threshold may remove many statistically significant variables.

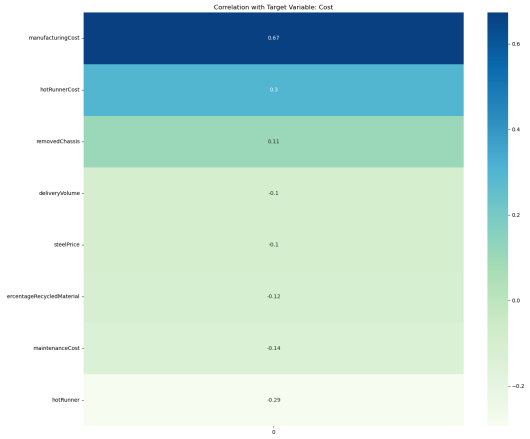
For **Cost**, the main influencing features were identified as *manufacturingCost*, *hotRunnerCost*, *hotRunner design time*, and *maintenanceCost*.

For **Human Health - Total**, the most important features included *nProdottiAnno*, *machineCycleTime*, *steelPrice*, *productionCost*, *deliveryVolume*, *deliveryPeriod*, and *manufacturingCost*, among others.

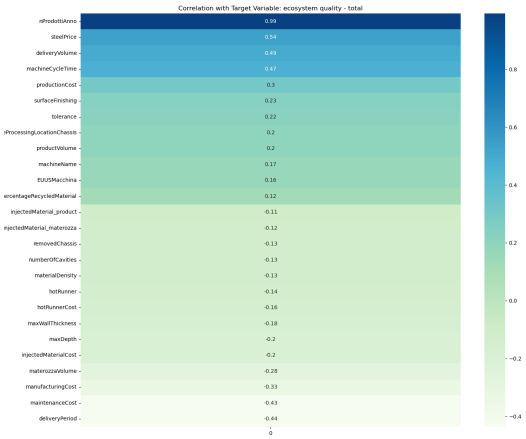
For **Resources** and **Ecosystem Quality - Total**, the major influencing features were *hotRunner*, *hotRunnerCost*, *manufacturingCost*, *designTime*, *removedChassis*, *surfaceProcessingLocationChassis*, *numberOfCavities*, *totalDistance*, *injectedMaterial_product*, *injectedMaterialCost*, and *percentageRecycledMaterial*.



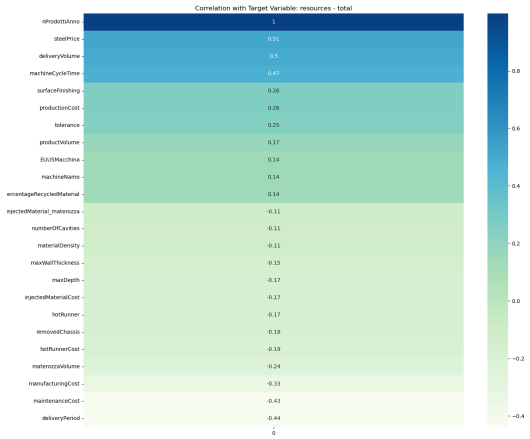
(a) Human Health - Total



(b) Cost



(c) Ecosystem Quality - Total



(d) Resources - Total

Figure 4.2: Pearson's correlation for different targets.

4.4.5 Final Filtration

Finally, the features have been selected for which the dcor score with the four target variables was greater than 0.1. This is a generous selection of thresholds. However, it eliminates the duplicates and empty features. Our final dataset consists of four target variables and 50 selected features.

5 Results

When the algorithm has been tested on a custom objective function to minimize the MRE and maximize the threshold, the results revealed that the algorithm performs a good job.

5.1 Multi Objective problems and VPBO

The optimizer was evaluated across five experimental runs with increasing trial budgets (1, 1, 10, and 20 trials; 5.1). The performance metric of interest was the Mean Relative Error (MRE), reported separately for each data split. The objective is to minimize the mean squared error and maximize the threshold, so a lower number of features are selected.

Run	Trials	Total Evals	Lowest MRE (per split)	Best Overall MRE	Total Time (min)
1	1	10	[0.24664, 0.197064, 0.313138, 0.011472]	0.011472	6.2
2	1	10	[0.173754, 0.075499, 0.288564, 0.002628]	0.002628	4.3
3	10	55	[0.147520, 0.062449, 0.234735, 0.002815]	0.002815	32.5
4	20	105	[0.174976, 0.061533, 0.175419, 0.000959]	**0.000959**	63.4

Table 5.1: VPBO Results: Multi-target MRE per split, best overall MRE, and total runtime.

5.1.1 Feature Count and Model Performance.

If we analyze from the graph below, when the number of features is high between 32 and 36, the objective score, MRE, is low at around 0.1. When the features are decreased to 17 as the threshold increases (e.g., to 17, 7, or 2 features), MRE generally worsens.

Similarly, as the threshold decreases, there is an increase in the hidden layer size of the neural network as shown by the diagonal trend 5.2. This indicates that excessive feature reduction impairs predictive

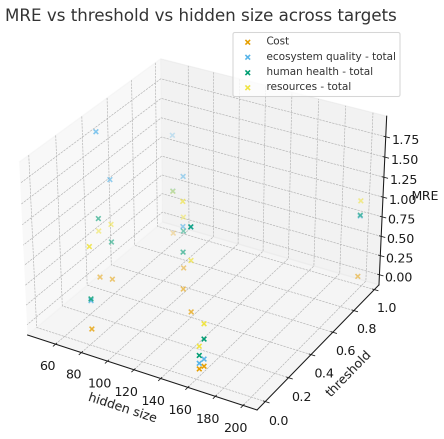


Figure 5.1: MRE vs Threshold vs Hidden layer numbers across all targets

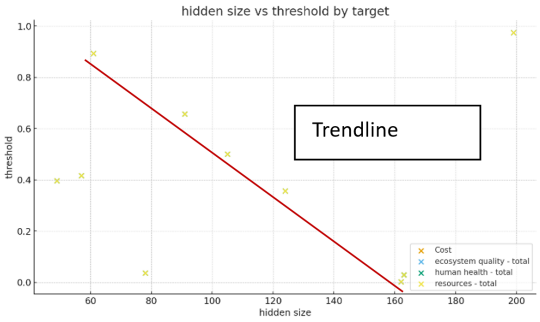


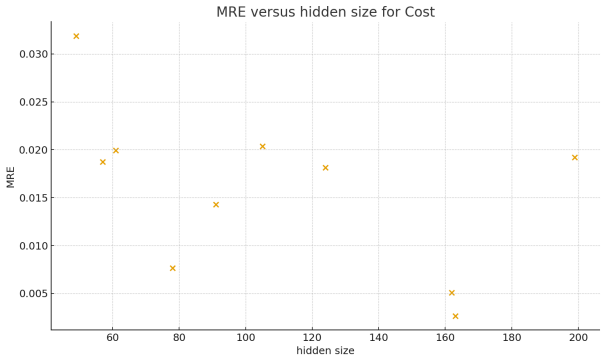
Figure 5.2: Hidden Layers and Model Capacity

performance, suggesting that the model relies on a broader set of features to capture the underlying data structure effectively.

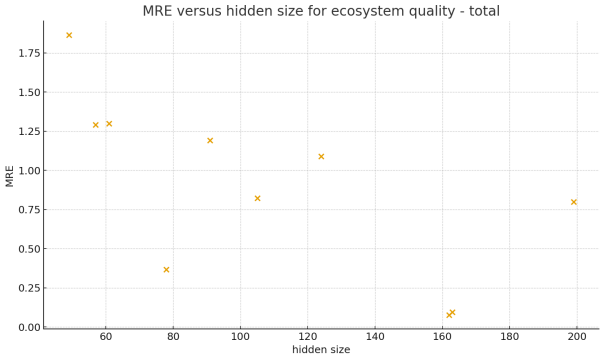
5.1.2 Hidden Size and Model Capacity.

The effect of hidden layer size on performance exhibits a clear trend. Larger hidden sizes (165) yield more stable and favorable MRE values across all targets, particularly when combined with moderate feature counts. Medium hidden sizes (70-120) maintain reasonable performance but demonstrate increased sensitivity when feature counts are reduced.

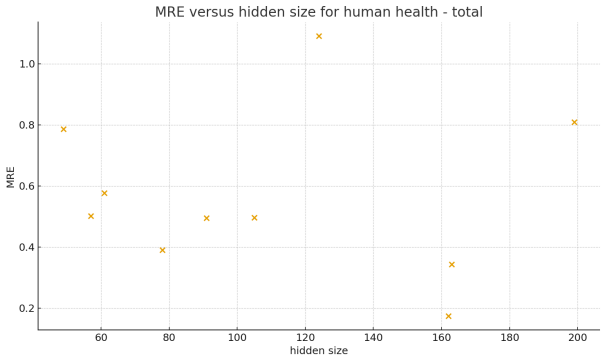
Conversely, very small hidden sizes (28, 15) result in sharp increases in MRE for ecosystem quality and human health-total, especially under conditions of aggressive feature selection. These findings suggest that insufficient model capacity leads to underfitting, with performance degrading rapidly when the feature space is constrained.



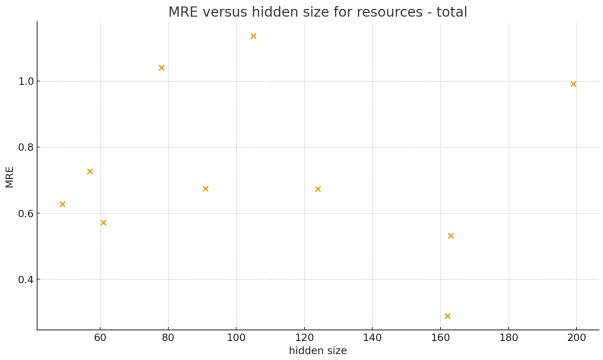
(a) MRE vs Hidden Layer Size: Cost



(b) MRE vs Hidden Layer Size: Ecosystem Quality - Total



(c) MRE vs Hidden Layer Size: Human Health - Total



(d) MRE vs Hidden Layer Size: Resources - Total

Figure 5.3: MRE as a function of hidden layer size for different targets.

5.1.3 Thresholding Effects.

The feature selection threshold (thr) significantly influences performance outcomes. Low thresholds (0.04–0.10) result in the retention of 32–36 features, producing the most favorable balance between MRE and score. Mid-range thresholds (0.20–0.40) substantially reduce the number of features, generally leading to performance degradation, though in some cases a slight improvement in MRE is observed, likely due to the removal of noisy or redundant features. High thresholds (> 0.50) reduce the feature count to as few as two, consistently resulting in degraded performance.

5.1.4 Consistency of Repeated Runs.

As expected, repeated experiments with identical configurations (same hidden size and k produce identical MRE and score values. This consistency confirms the reproducibility of the observed trends under controlled experimental conditions.

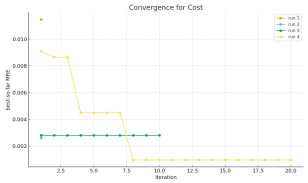
Even to decrease the number of features to 32, the number of hidden layers to 28, and the score to 0.04, with a corresponding MRE of 0.00488.

5.1.5 Effect of Number of Trials

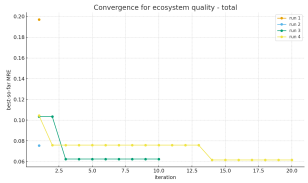
The number of trials represents the iterations required for the Bayesian Optimization (BO) algorithm to converge towards the minimum mean relative error (MRE). To evaluate this effect, the number of trials progressively increased from 1 to 20 while keeping other parameters constant.

On individual targets, the best MRE vs trial has been plotted. For the cost, the algorithm converges in the start and maintains the same MRE in the following iterations. The run with 20 iterations explores the search space more by gradually decreasing MRE. However, we see a convergence after 8 iterations. The same can be said about resources-total, with similar convergence at 8 trials.

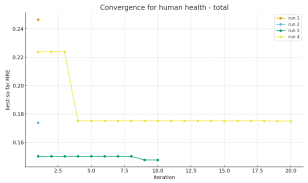
However, for ecosystem quality and human health, a lower number of trials actually leads to a lower MRE. However, the MRE converges after 14 trials in the case of ecosystem quality, indicating higher exploration.



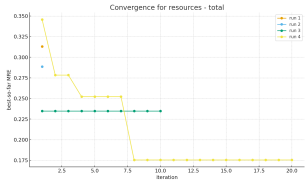
(a) Convergence: Cost



(b) Convergence: Ecosystem Quality - Total



(c) Convergence: Human Health - Total



(d) Convergence: Resources - Total

Figure 5.4: Convergence plots for different targets.

Single-trial runs (Run 1 and Run 2) showed rapid convergence, with total wall-clock times of approximately 6.2 minutes and 4.3 minutes, respectively. Best final MRE values ranged from 0.0115 (Run 1) to 0.0026 (Run 2), indicating strong error reduction from the initial estimates ($\sim$ 0.25–0.30). These results demonstrate that even limited exploration of the hyperparameter space can yield high-quality solutions.

In most cases, the best performance was achieved within only 10 trials, suggesting that the algorithm converges rapidly. However, increasing the number of trials beyond 10 results in only slight improvements in accuracy. This behavior can be attributed to the exploratory nature of Bayesian Optimization, which deliberately samples a broader set of candidate points. While such exploration is beneficial in multimodal optimization problems, it may slightly reduce accuracy when the solution landscape is relatively well-defined.

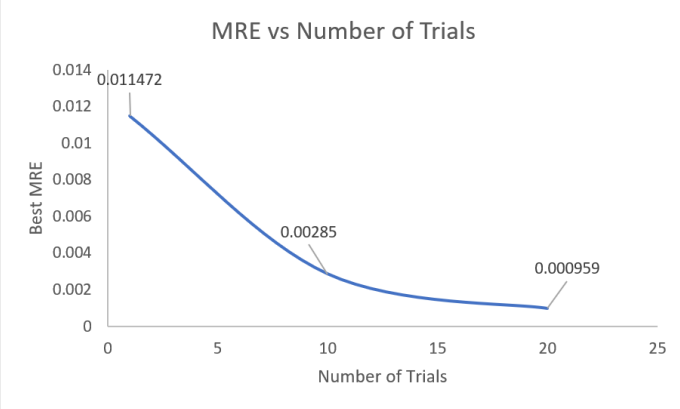


Figure 5.5: MRE and Number of Trials

Overall, these findings suggest that a moderate number of trials (≈ 10–20) strikes an optimal balance between convergence speed and predictive accuracy.

5.1.6 Efficiency of Improvement

When comparing efficiency in terms of MRE reduction per minute, the single-trial runs were the most efficient. For example, Run 2 achieved a reduction of 0.297 in MRE over 4.3 minutes, whereas Run 4 achieved 0.299 reduction over more than 63 minutes. Thus, while longer runs can yield superior absolute results, they do so with much lower marginal efficiency.

This trade-off highlights the importance of balancing exploration depth with computational constraints. For applications where near-optimal solutions are sufficient, short single-trial runs already provide excellent performance. Conversely, for high-stakes applications where the lowest possible error is critical, extended optimization (e.g., Run 4) can justify the additional cost.

5.2 Comparison with GridSearchCV

As shown in Table 5.2, GridSearchCV achieves the lowest error on all four targets relative to VPBO: Cost (MRE 0.0011 vs. 0.0028, ∼60.9% lower), Human Health (0.1132 vs. 0.1475, ∼23.3% lower), Ecosystem-Total (0.0469 vs. 0.0624, ∼24.9% lower), and Resources (0.1735 vs. 0.2347, ∼26.1% lower). Averaged across targets, GridSearchCV reduces mean MRE from 0.1119 (VPBO) to 0.0837 (−25.2%). This accuracy gain, however, comes with a higher search cost: GridSearchCV used 220 evaluations and 3394 s (≈56.6 min), whereas VPBO completed in 55 evaluations and 1949.8 s (≈32.5 min), i.e., ∼4× fewer evaluations and ∼1.74× less wall-clock time. Accordingly, GridSearchCV is preferable when minimizing prediction error is paramount and additional compute is acceptable; VPBO remains attractive when evaluation budget or time is constrained. Overall, the extra search budget suggests VPBO is sufficient when marginal accuracy improvements do not justify additional compute.

	GridSearchCV (220 evals, 3394 s)			VPBO (55 evals, 1949.8 s)		
Target	**Thr**	**MRE**	**Hidden**	**Thr**	**MRE**	**Hidden**
Cost	0.000	0.0011	10	0.019	0.0028	200
Human Health	0.100	0.1132	100	0.027	0.1475	161
Ecosystem-Total	0.100	0.0469	50	0.022	0.0624	165
Resources	0.000	0.1735	50	0.216	0.2347	79

Table 5.2: Compact comparison of GridSearchCV and VPBO by target with fixed column widths to prevent overflow.

5.2.1 Final Selected Features, Threshold, and Hidden Sizes for All Target

Overall, the VPBO configurations yield low error with compact search effort and coherent feature sets across targets. Small dCor thresholds were optimal for Human health (0.027) and Ecosystem quality (0.022), indicating diffuse signal across many design and process variables, whereas Resources benefited from a higher threshold (0.216), suggesting a sparser, more selective driver set; Cost also preferred a small threshold (0.019) yet retained a broad feature subset, consistent with its economic coupling to multiple upstream parameters. The achieved MREs—0.1475 (Human health), 0.0624 (Ecosystem), 0.2347 (Resources), and 0.0028 (Cost)—are competitive while requiring far fewer evaluations than exhaustive grid search, making VPBO a pragmatic choice under big-data constraints. Notably, recurrent features such as manufacturingCost, machineCycleTime, productionCost, steelPrice, and schedule/logistics factors (setupTime, deliveryVolume/Period) dominate multiple targets, whereas the Resources model concentrates on a smaller subset, aligning with its higher threshold. These results support using VPBO for efficient hyperparameter selection and threshold tuning, with future work focusing on multivariate feature interactions and stability analyses to validate robustness beyond univariate dCor filtering.

Target	Threshold	Hidden	MRE	Valid for	Selected features (dCor ≥ thr)
Human health - total	0.027	161	0.1475	Photochemical oxidation; ionising radiation; respiratory effects; human toxicity; ozone layer depletion.	surfaceProcessingLocationCavity, hotRunner, hotRunnerCost, manufacturingCost, mouldValidationCost, designTime, hourlyDesignCost, removedChassis, surfaceProcessingLocationChassis, removedCavity, numberOfCavities, totalDistance, injectedMaterial_product, injectedMaterialCost, percentageRecycledMaterial, maxDepth, maxWallThickness, productVolume, materozzaVolume, nProdottiAnno, materialDensity, tolerance, surfaceFinishing, machineCycleTime, maintenanceCost, productionCost, injectedMaterial_materozza, injectionMouldingProcess, steelPrice, mouldMaterialName, machineName, EUUSMacchina, CNMacchina, setupTime, deliveryVolume, deliveryPeriod
Ecosystem quality - total	0.022	165	0.0624	Terrestrial ecotoxicity; aquatic ecotoxicity; land occupation; acidification & nutrification; total climate change impact.	surfaceProcessingLocationCavity, hotRunner, hotRunnerCost, manufacturingCost, mouldValidationCost, designTime, hourlyDesignCost, removedChassis, surfaceProcessingLocationChassis, removedCavity, numberOfCavities, totalDistance, injectedMaterial_product, injectedMaterialCost, percentageRecycledMaterial, maxDepth, maxWallThickness, productVolume, materozzaVolume, nProdottiAnno, materialDensity, tolerance, surfaceFinishing, machineCycleTime, maintenanceCost, productionCost, injectedMaterial_materozza, injectionMouldingProcess, steelPrice, mouldMaterialName, machineName, EUUSMacchina, CNMacchina, setupTime, deliveryVolume, deliveryPeriod
Resources - total	0.216	79	0.2347	Mineral extraction; non-renewable energy use; total resources.	manufacturingCost, materozzaVolume, nProdottiAnno, tolerance, surfaceFinishing, machineCycleTime, maintenanceCost, productionCost, steelPrice, setupTime, deliveryVolume, deliveryPeriod
Cost	0.019	200	0.0028	Cost.	surfaceProcessingLocationCavity, hotRunner, hotRunnerCost, manufacturingCost, mouldValidationCost, designTime, hourlyDesignCost, removedChassis, surfaceProcessingLocationChassis, removedCavity, numberOfCavities, totalDistance, injectedMaterial_product, injectedMaterialCost, percentageRecycledMaterial, maxDepth, maxWallThickness, productVolume, materozzaVolume, nProdottiAnno, materialDensity, tolerance, surfaceFinishing, machineCycleTime, maintenanceCost, productionCost, injectedMaterial_materozza, injectionMouldingProcess, steelPrice, mouldMaterialName, machineName, EUUSMacchina, CNMacchina, setupTime, deliveryVolume, deliveryPeriod

Table 5.3: VPBO final configurations with domain applicability and selected features per target.

Conclusion

The proposed workflow meets the professor's brief. It identifies the GRETA input parameters that most influence sustainability and cost, and it does so with a computationally efficient search. Correlation analysis compresses sixteen outputs into four operational targets that align with GRETA reporting. Distance correlation and Pearson correlation remove duplicates and weak predictors. Variable partitioned Bayesian optimization then tunes the feature threshold and network capacity to balance accuracy across all targets. The method converges in about ten trials and requires far fewer evaluations than exhaustive search. Performance degrades when too few features are kept, while larger hidden sizes stabilize the error. Final MRE values are competitive, and the comparison with GridSearchCV shows the expected trade-off: slightly better accuracy at a higher evaluation cost. Recurrent high-impact drivers across targets include manufacturing cost, machine cycle time, production cost, steel price, setup time, and logistics volumes and periods, which give GRETA users clear levers for design and process changes. Future work should test multivariate feature interaction filters, add uncertainty quantification, and explore multi-fidelity references inside the optimizer. Integration into GRETA can expose interactive sensitivity and permit user steer over accuracy versus speed. Overall, the system delivers actionable feature importance for sustainability assessment with strong efficiency and sound validation.

List of Figures

List of Tables

Bibliography

[1] S. A. Bello, L. O. Oyedele, O. O. Akinade, M. Bilal, J. M. D. Delgado, L. Akanbi, and A. O. Ajayi. Cloud-enabled machine learning for predictive environmental impact assessment of construction projects. *Journal of Cleaner Production*, 268:122–157, 2020.

[2] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems*, volume 24, pages 2546–2554. NeurIPS, 2011.

[3] A. Biswas, A. N. Morozovska, M. Ziatdinov, E. A. Eliseev, and S. V. Kalinin. Multi-objective bayesian optimization of ferroelectric materials with interfacial control for memory and energy storage applications. *Journal of Applied Physics*, 130(20):204102, 2021.

[4] Eric Brochu, Vlad M. Cora, and Nando de Freitas. A tutorial on bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. *arXiv preprint arXiv:1012.2599*, 2010.

[5] Marc Claesen and Bart De Moor. Hyperparameter search in machine learning. *arXiv preprint arXiv:1502.02127*, 2015.

[6] S. Cucurachi, C. van der Giesen, and J. Guinée. Ex-ante lca of emerging technologies. *Nature Communications*, 10(1):1509, 2019.

[7] Roman Garnett. *Bayesian Optimization*. Cambridge University Press, 2021.

[8] L. D. González and V. M. Zavala. New paradigms for exploiting parallel experiments in bayesian optimization. *Computers and Chemical Engineering*, 170:108110, 2023.

[9] Zena M. Hira and Duncan F. Gillies. A review of feature selection and feature extraction methods applied on microarray data. *Advances in Bioinformatics*, 2015:1–13, 2015. Article ID 198363.

[10] Jianyu Miao and Ling Niu. A survey on feature selection. *Procedia Computer Science*, 91:919–926, 2016.

[11] Jonas Mockus. *Bayesian Approach to Global Optimization: Theory and Applications*, volume 37. Springer Science & Business Media, 2012.

[12] F. Müller, M. Hiete, and J. Wehling. Machine learning in life cycle assessment: A review of current status and future perspectives. *Journal of Industrial Ecology*, 25(6):1570–1587, 2021.

[13] E.-D. Şandru and E. David. Unified feature selection and hyperparameter bayesian optimization for machine learning based regression. In *2019 International Symposium on Signals, Circuits and Systems (ISSCS)*, pages 1–4. IEEE, 2019.

[14] Joseph A. Selekman, J. Qiu, K. Tran, J. Stevens, V. Rosso, E. Simmons, Y. Xiao, and J. Janey. High-throughput automation in chemical process development. *Annual Review of Chemical and Biomolecular Engineering*, 8:525–547, 2017.

[15] Jasper Snoek, Oren Rippel, Kevin Swersky, Ryan Kiros, Nadathur Satish, Narayanan Sundaram, Md Mostofa Ali Patwary, Mostofa Ali, Mr Prabhat, and Ryan P. Adams. Scalable bayesian optimization using deep neural networks. *Proceedings of the 32nd International Conference on Machine Learning*, 37:2171–2180, 2015.

[16] Gábor J Székely, Maria L Rizzo, and Nail K Bakirov. Measuring and testing dependence by correlation of distances. *The Annals of Statistics*, 35(6):2769–2794, 2007.

[17] S. Teso, F. Bilo, and A. Passerini. Explainable ai for sustainability: A survey. *AI and Ethics*, 1(2):1–23, 2021.